IT342 · Systems Integration & Architecture 1

MediBook
Phase 2 Report

Health Appointment & Records Integration System. Project development continuation across Web, Mobile, and Backend.

Spring Boot ReactJS Android · Kotlin Supabase · PostgreSQL JWT + BCrypt OpenFDA

Prepared by: **Gyle M. Amihan**

Submission: **Phase 2 · Development Continuation**

Date: **July 7, 2026**

Repository: **github.com/rig-gg/MediBook**

SECTION 1

GitHub Commit History

Each feature ships as its own commit. Commits are grouped by layer below; hashes and messages are pulled from main. Replace the dates with your local commit dates if they differ, and fill the blank hashes after committing the health-records and UI-fix work.

Repository: https://github.com/rig-gg/MediBook

Backend **Appointments & Scheduling**

HASH	MESSAGE	DATE
c220735	feat(backend): add Appointment entity with unique schedule constraint and AppointmentStatus enum	Jul 7, 2026
2d34c4b	feat(backend): add DoctorSchedule entity and repository with pessimistic locking	Jul 7, 2026
0d50bcd	feat(backend): add DoctorScheduleRequest/Response DTOs	Jul 7, 2026
c34783b	feat(backend): add DoctorScheduleService with overlap prevention (BR-003)	Jul 7, 2026

HASH	MESSAGE	DATE
41aec90	feat(backend): add DoctorScheduleController for schedule creation and listing	Jul 7, 2026
ec98967	feat(backend): add AppointmentRepository with patient and status queries	Jul 7, 2026
fe00cb0	feat(backend): add AppointmentRequest/Response DTOs	Jul 7, 2026
8c108fe	feat(backend): add AppointmentService with transactional conflict-safe booking (FR-006)	Jul 7, 2026
ab9c742	feat(backend): add AppointmentController for booking, history, and status updates	Jul 7, 2026
d9e1ba9	feat(backend): restrict schedule/appointment endpoints by role in SecurityConfig	Jul 7, 2026
bbb2575	fix(backend): catch JwtException in JwtAuthFilter to prevent login crash on expired tokens	Jul 7, 2026

Backend

Health Records

HASH	MESSAGE	DATE
2716b44	feat(backend): add HealthRecord entity with immutable one-per-appointment constraint (BR-004)	Jul 7, 2026
ee6e58e	feat(backend): add HealthRecordRepository + Request/Response DTOs	Jul 7, 2026
5a9b575	feat(backend): add HealthRecordService completing appointment on record creation (FR-007)	Jul 7, 2026
c98e0df	feat(backend): add HealthRecordController with DOCTOR-only write, STAFF/DOCTOR read (FR-008)	Jul 7, 2026
926d798	feat(backend): add DoctorResponse DTO to avoid exposing entity/password data and specialization search query to DoctorRepository	Jul 7, 2026

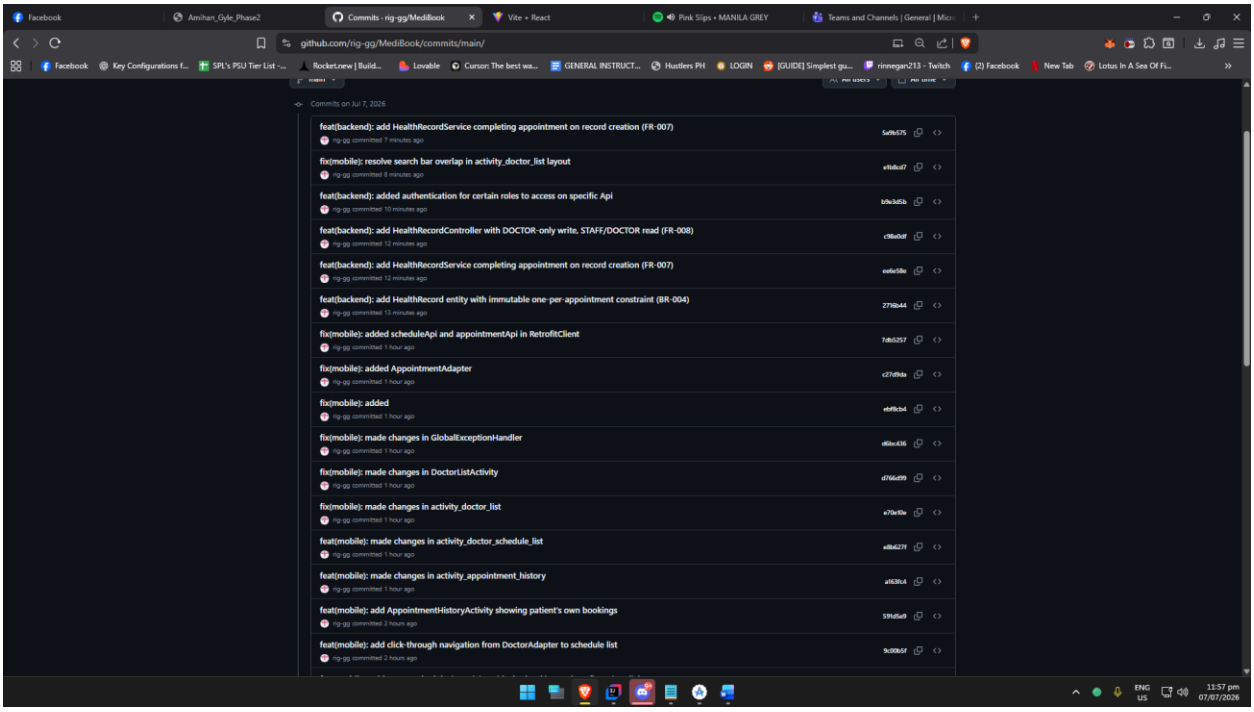
Web ReactJS Dashboard

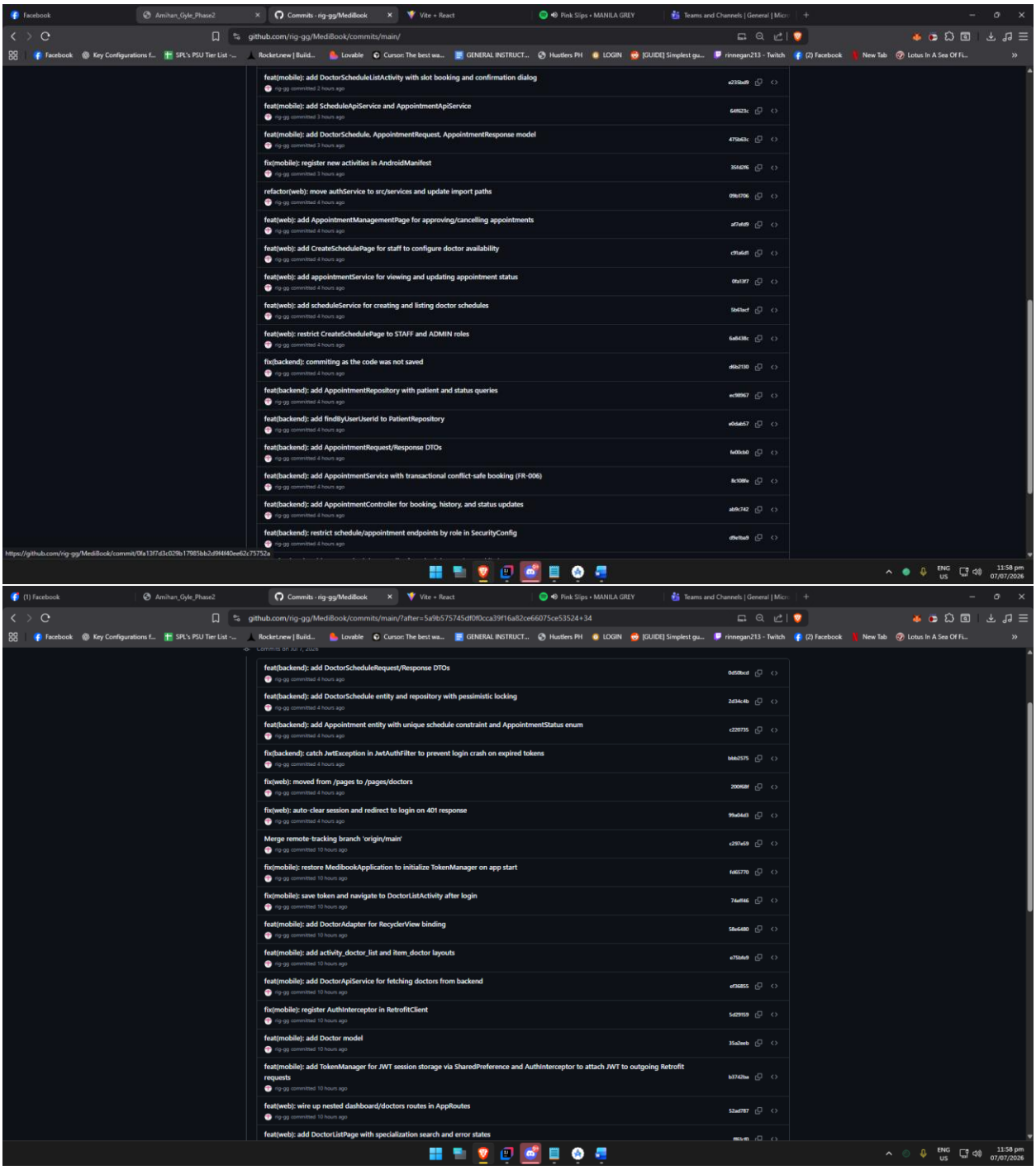
HASH	MESSAGE	DATE
af7efd9	feat(web): add AppointmentManagementPage for approving/cancelling appointments	Jul 7, 2026
c91a6d1	feat(web): add CreateSchedulePage for staff to configure doctor availability	Jul 7, 2026
0fa13f7	feat(web): add appointmentService for viewing and updating appointment status	Jul 7, 2026
5b61acf	feat(web): add scheduleService for creating and listing doctor schedules	Jul 7, 2026
6a8438c	feat(web): restrict CreateSchedulePage to STAFF and ADMIN roles	Jul 7, 2026
09b1706	refactor(web): move authService to src/services and update import paths	Jul 7, 2026
99a04d3	fix(web): auto-clear session and redirect to login on 401 response	Jul 7, 2026
200f68f	fix(web): moved from /pages to /pages/doctors	Jul 7, 2026

mobile Android · Kotlin

HASH	MESSAGE	DATE
475b63c	feat(mobile): add DoctorSchedule, AppointmentRequest, AppointmentResponse model	Jul 7, 2026
64f623c	feat(mobile): add ScheduleApiService and AppointmentApiService	Jul 7, 2026
e235bd9	feat(mobile): add DoctorScheduleListActivity with slot booking and confirmation dialog	Jul 7, 2026
9c00b5f	feat(mobile): add click-through navigation from DoctorAdapter to schedule list	Jul 7, 2026
591d5e9	feat(mobile): add AppointmentHistoryActivity showing patient's own bookings	Jul 7, 2026
c27d9da	fix(mobile): added AppointmentAdapter	Jul 7, 2026
7db5257	fix(mobile): added scheduleApi and appointmentApi in RetrofitClient	Jul 7, 2026
35fd2f6	fix(mobile): register new activities in AndroidManifest	Jul 7, 2026

HASH	MESSAGE	DATE
e1b8cd7	fix(mobile): resolve search bar overlap in activity_doctor_list layout	Jul 7, 2026





SECTION 2

Implementation Explanation Report

This report covers the features that are working and demonstrable in this build. Scheduling and appointment booking are implemented in the backend and clients but not yet

demonstrated end-to-end, since no doctor schedules exist in the database yet; they are noted as the next step.

Working now: Role-based auth & navbar · doctor directory with live count (Staff/Doctor) · patient doctor list (Mobile)

Built, pending: dataSchedule creation & patient booking, once slots are seeded

Role-Based Login & Dashboard Navigation

FR-001 · FR-002

Purpose

Authenticate users with JWT and show a dashboard whose navbar adapts to the signed-in role. Staff and Doctor land on the dashboard with Dashboard and Doctors in the navbar; the JWT drives what is visible.

React component

LoginPage → DashboardPage with a role-aware Navbar; authService stores the token and current user

Spring Boot

AuthController · AuthService · JwtAuthFilter · JwtUtil · SecurityConfig · User entity

Endpoints

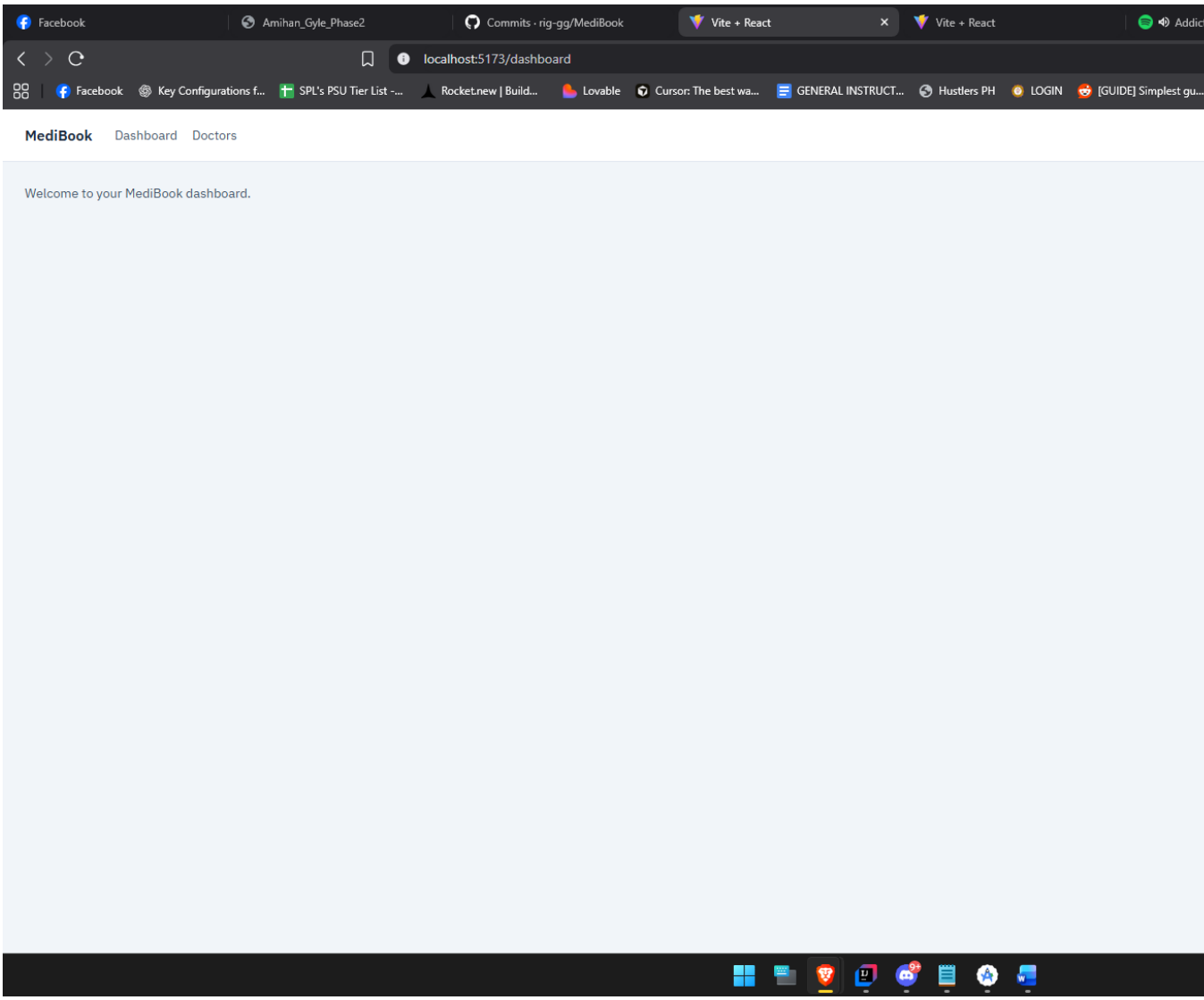
POST /api/auth/login (returns JWT) · POST /api/auth/register

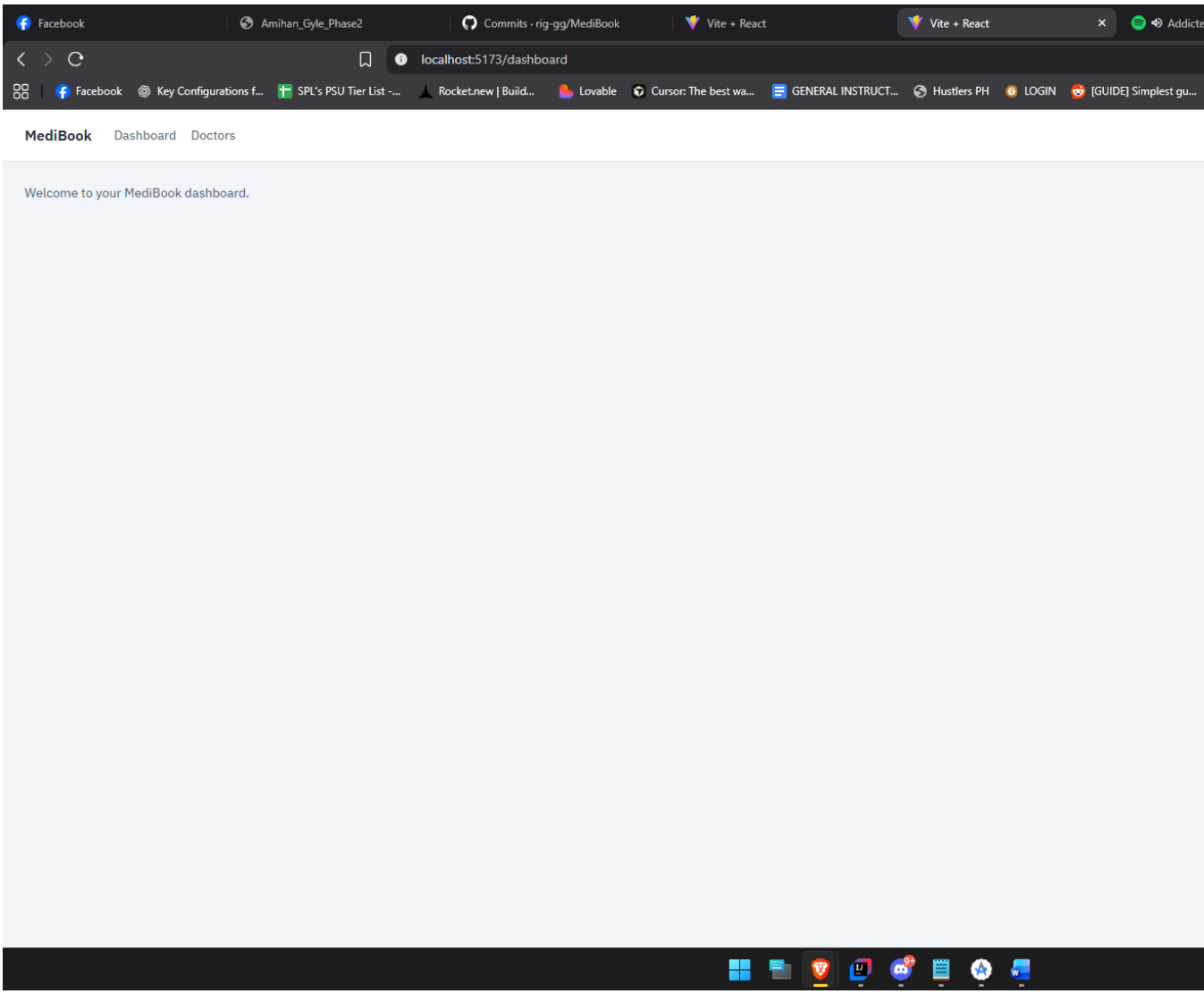
DB table

users (role column: PATIENT / STAFF / DOCTOR / ADMIN)

Flow: login to role-based dashboard

1. User submits credentials on LoginPage → POST /api/auth/login.
2. AuthService checks the BCrypt hash and returns a signed JWT carrying the role.
3. authService stores the token; the Navbar renders Dashboard + Doctors based on the role claim.
4. Every later request carries the JWT; JwtAuthFilter validates it and SecurityConfig enforces role access.


Facebook
Amihan_Gyle_Phase2
Commits · rig-gg/MediBook
Vite + React
Vite + React
localhost:5173/dashboard
Facebook
Key Configurations f...
SPL's PSU Tier List -...
Rocket.new | Build...
Lovable
Cursor: The best wa...
GENERAL INSTRUCT...
Hustlers PH
LOGIN
[GUIDE] Simplest gu...
MediBook
Dashboard
Doctors
Welcome to your MediBook dashboard.



Doctor Directory & Count (Web)

Purpose

Staff and Doctor open the Doctors page from the navbar and see the list of doctors along with the total doctor count. The backend returns a DTO instead of the raw entity to avoid serialization failures.

React component

DoctorsPage (renders the list + count) using a doctor service call

Spring Boot

DoctorController · DoctorService (@Transactional read) · DoctorRepository · DoctorResponse DTO

Endpoint

GET /api/doctors

DB tables

doctors, users

Error handling

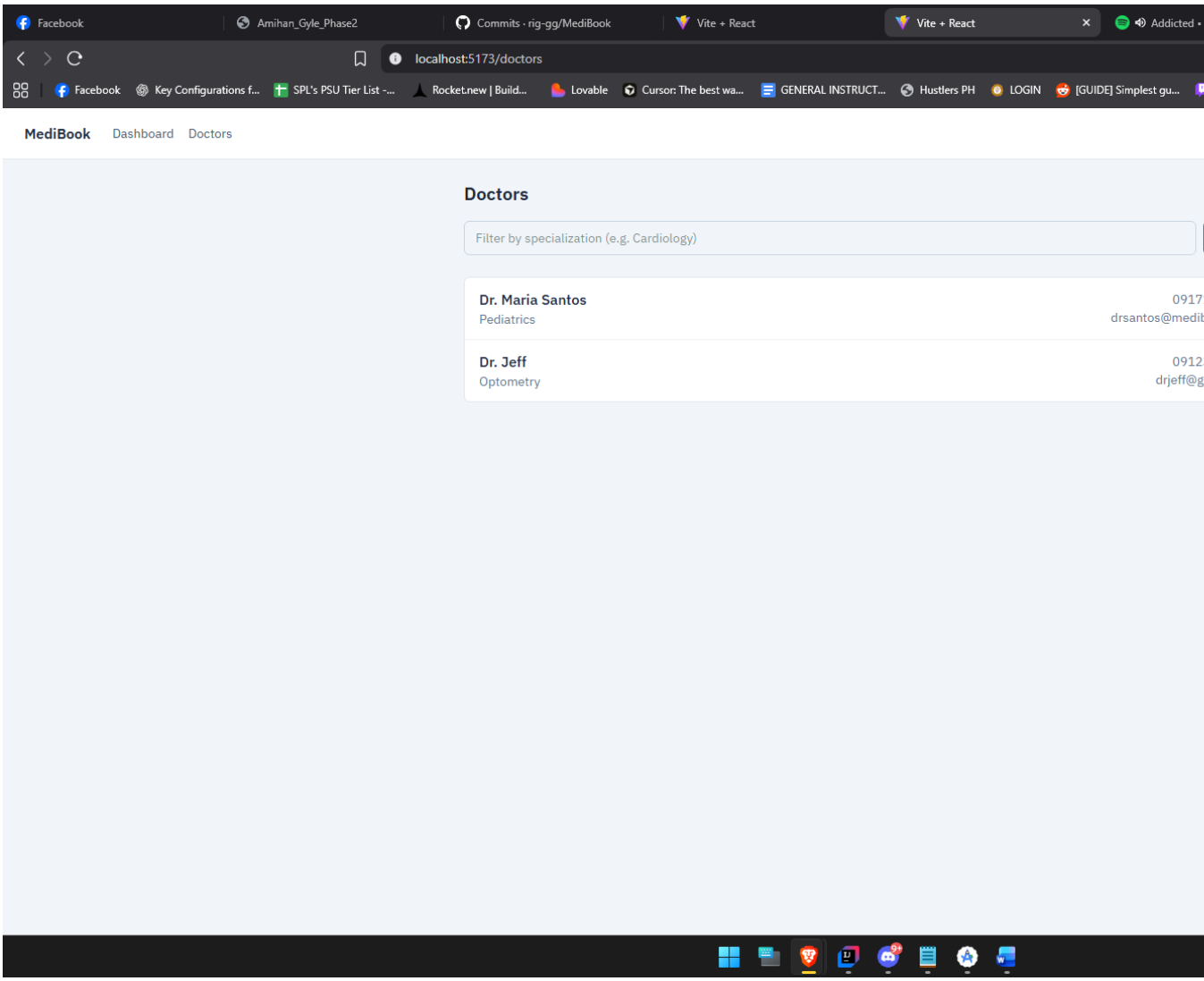
GlobalExceptionHandler maps 404 / 400 / 409 / 500 to structured JSON, so the frontend shows a clean message instead of a raw stack trace.

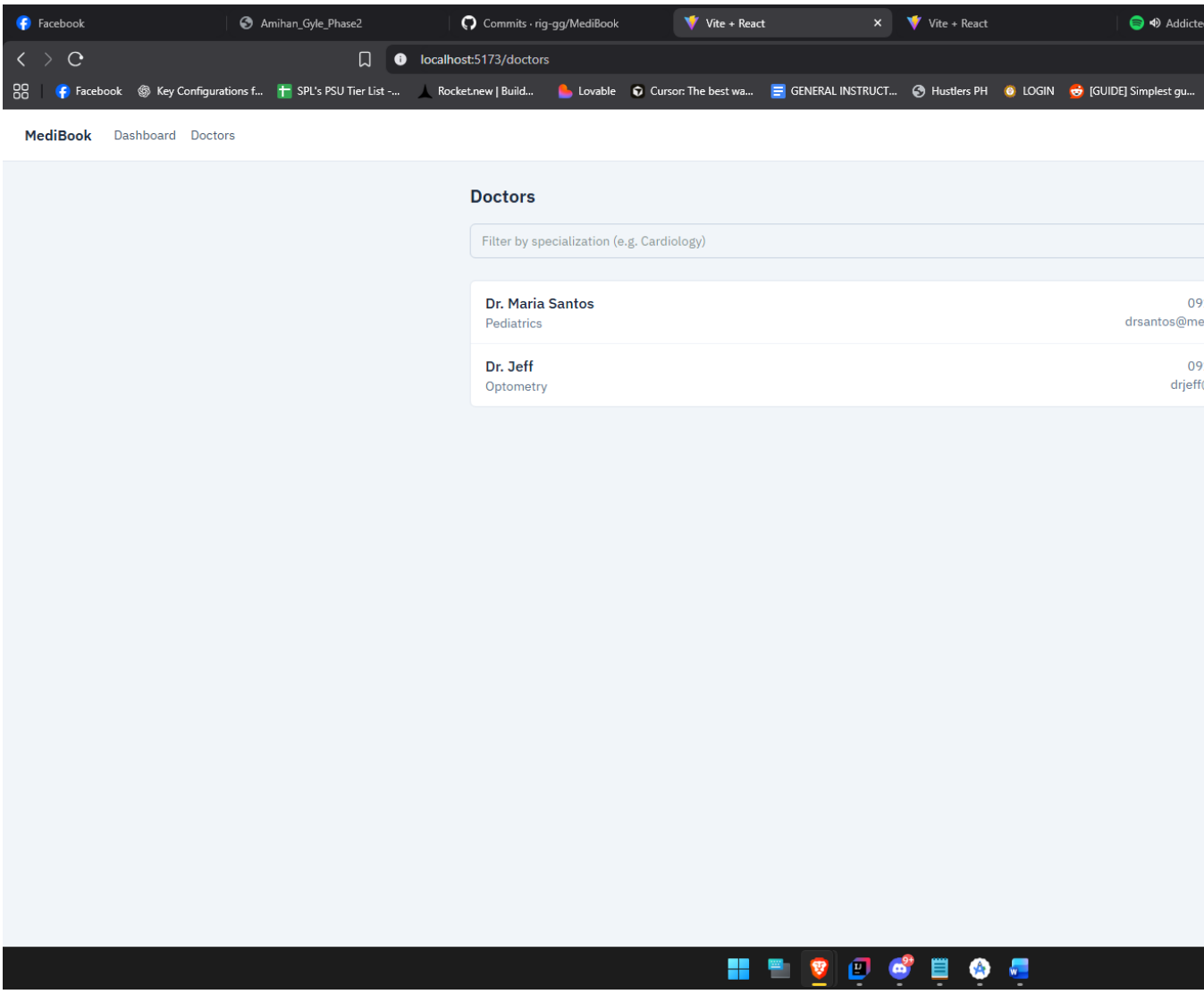
Flow: view doctors and count

1. Staff/Doctor clicks Doctors in the navbar → DoctorsPage calls GET /api/doctors with the JWT.
2. DoctorService maps each Doctor to a DoctorResponse inside a transaction (safe lazy email load).
3. The page renders the doctor list and derives the count from the returned array length.

DoctorResponse.java - flattens Doctor + User email into a safe DTO

DoctorService.java - @Transactional read, entity → DTO mapping


Facebook
Amihan_Gyle_Phase2
Commits · rig-gg/MediBook
Vite + React
Vite + React
Addicted
localhost:5173/doctors
Facebook
Key Configurations f...
SPL's PSU Tier List -...
Rocket.new | Build...
Lovable
Cursor: The best wa...
GENERAL INSTRUCT...
Hustlers PH
LOGIN
[GUIDE] Simplest gu...
MediBook
Dashboard
Doctors
Doctors
Filter by specialization (e.g. Cardiology)
Dr. Maria Santos
Pediatrics
0917
drsantos@medib
Dr. Jeff
Optometry
0912
drjeff@g



Patient Doctor List (Android Mobile)

FR-003 (view)
consistency

Purpose

Authenticated patients open the app and browse the same doctors served by the backend, with a specialization filter. Because no schedules are seeded yet, booking is not yet exercised; the list itself confirms web/mobile read the same source of truth.

Android screen

DoctorListActivity · DoctorAdapter · activity_doctor_list.xml

Networking

RetrofitClient · DoctorApiService · AuthInterceptor (attaches JWT) · Doctor model

Endpoint

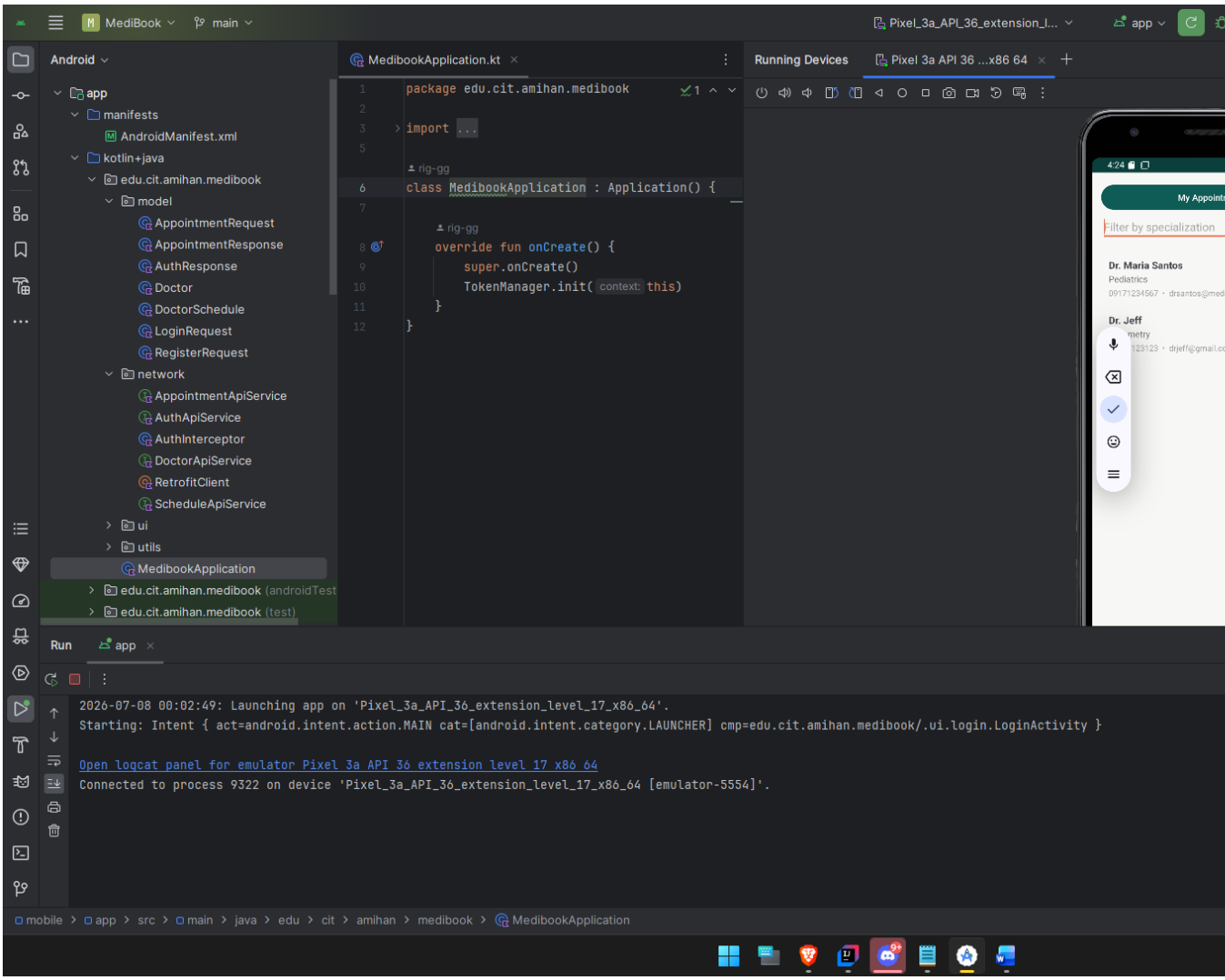
GET /api/doctors (same endpoint as web)

DB tables

doctors, users

Flow: patient views doctors

1. Patient logs in on mobile; AuthInterceptor attaches the JWT to every call.
2. DoctorListActivity calls GET /api/doctors via Retrofit.
3. DoctorAdapter binds the results into the RecyclerView; the specialization field filters the list.
4. Same data as the web Doctors page, confirming consistency across clients (requirement 4).



Next Step: Schedules & Booking

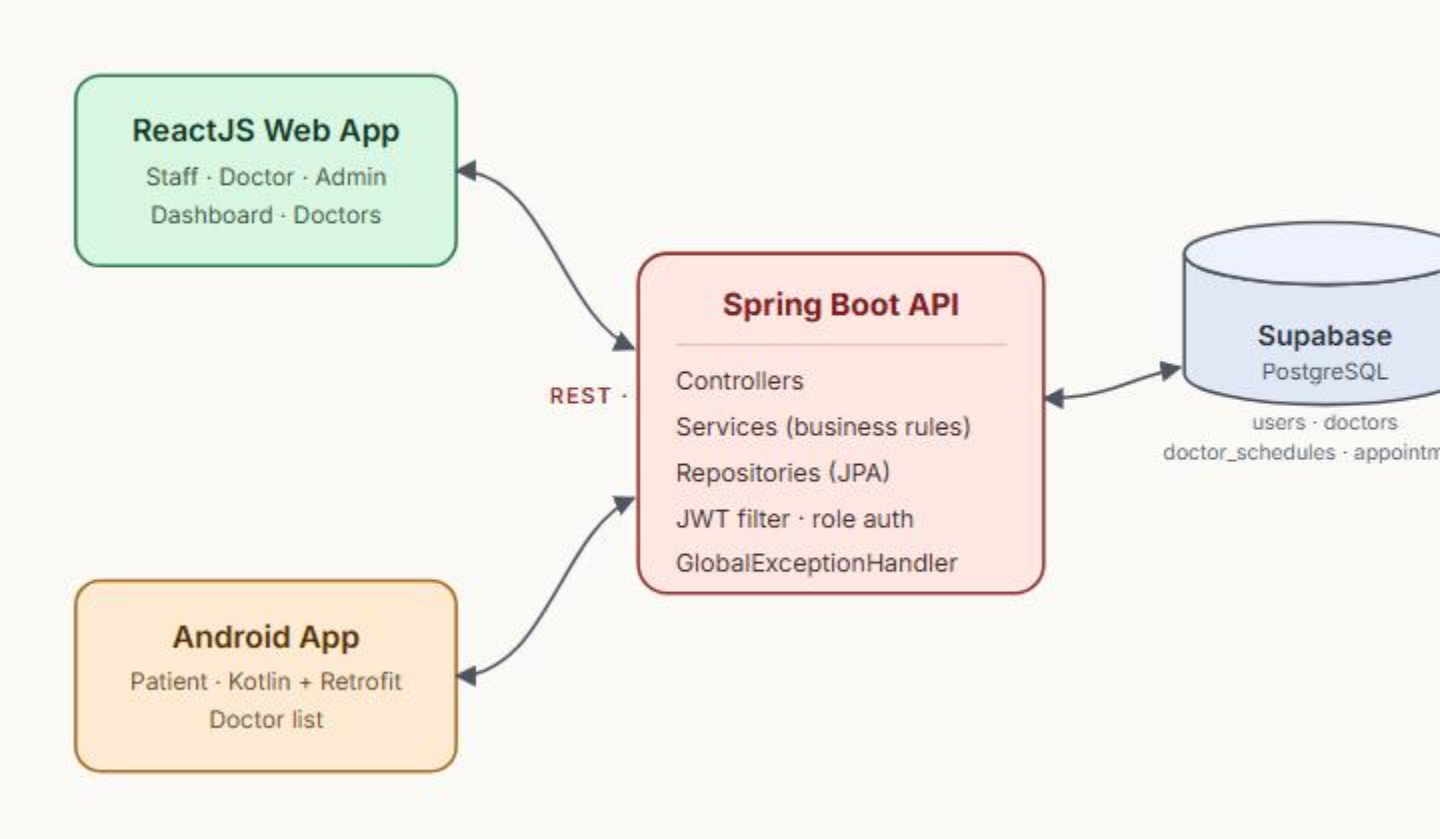
FR-004 · FR-005
FR-006 · BR-003

The scheduling and booking layer is already implemented across all three tiers (DoctorScheduleService with overlap prevention, AppointmentService with conflict-safe booking, the web CreateSchedulePage, and the mobile DoctorScheduleListActivity). Per FR-005, Staff/Admin create doctor availability; once slots are seeded, patients book against them. This is the immediate next milestone and is not claimed as demonstrated in this submission.

SECTION 3

System Architecture Diagram

How the two clients, the REST API, the database, and the external services connect.



Request flow. The ReactJS web app (Staff/Doctor/Admin) and the Android app (Patient) both talk to the same Spring Boot REST API over HTTPS, sending a JWT on every authenticated call. Controllers pass work to Services, where the business rules live; Services use JPA Repositories to read and write the **Supabase PostgreSQL** tables. Because both clients hit the same GET /api/doctors endpoint, the doctor data stays consistent across web and mobile. A single GlobalExceptionHandler turns failures into structured JSON so both clients handle errors the same way. Schedule and appointment tables exist and are wired; they populate once Staff seed availability.